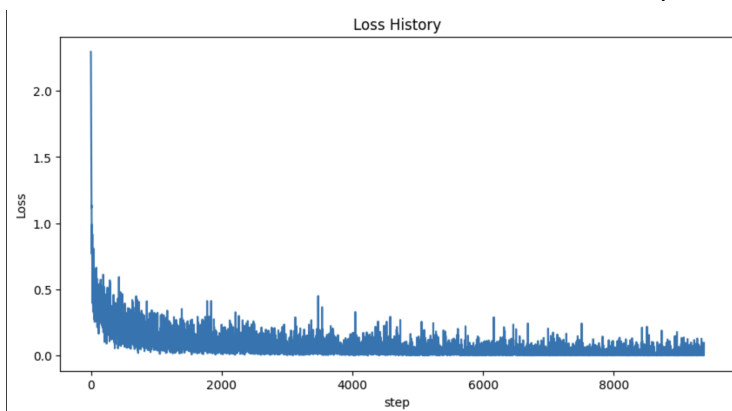


Brandon Chau
028171804
CECS 458
Dr. Arash Sarshar

Report

I chose my model's output to be logits instead of applying softmax to the output. In my forward function, "softmax()" was not applied in the final layer, "output2 = self.l2(input_2)." Based on this choice, I used the "CrossEntropyLoss()" function, as the function expects an input of logits. "CrossEntropyLoss()" also applies its own softmax before its loss calculation. If I were to softmax the logits before calling the function, it would produce incorrect training behaviors due to double softmax.

During training, I was able to record the loss value at each step by appending the output to "loss_history[]". The plot was able to show the loss value decreasing and converging. As the task states to change the optimizer to Adam from SGD, I experimented with the learning rate and found that a lower rate works best with Adam, producing low loss values.



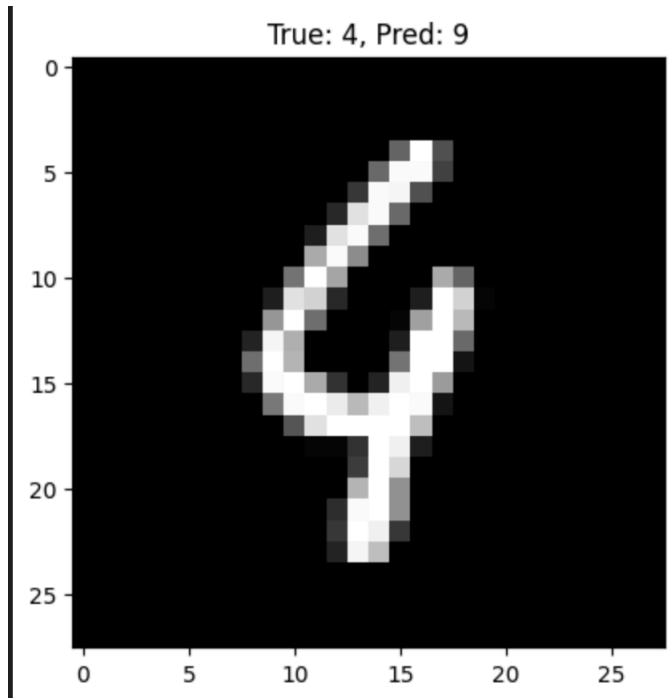
I trained the model for 10 epochs:

```
Epoch: 0 loss value: 0.1946
Epoch: 1 loss value: 0.0530
Epoch: 2 loss value: 0.0078
Epoch: 3 loss value: 0.1178
Epoch: 4 loss value: 0.0092
Epoch: 5 loss value: 0.0144
Epoch: 6 loss value: 0.0143
Epoch: 7 loss value: 0.0311
Epoch: 8 loss value: 0.0541
Epoch: 9 loss value: 0.0962
```

The Adam optimizer with a learning rate of 0.001 was able to have highly accurate classifications:

```
Batch Accuracy: 0.9790
Overall Accuracy: 0.9787
```

In order to obtain the misclassified image, I created the function “find_misclassified,” in which the structure is similar to the training function. However, instead, it locates and returns the indices of the input image, the true label, the model’s predicted label, and the predicted softmax probabilities. Below is the output of the misclassified image as well as the softmax probabilities:



```
Predicted probabilities: [7.4741223e-07 3.0658057e-07 3.4416064e-06 4.0766422e-06 9.7385474e-02  
9.4132574e-06 1.5424511e-04 2.8982060e-06 1.6514181e-05 9.0242279e-01]
```